Call Log Merger: Algorithms

This document describes the doubly linked list program that reads two call logs, merges them, searches and deletes callers, and reports call statistics. Each function is written as language-neutral pseudocode, followed by a complexity table.

1. Data Structure

Each call is a Node with fields **name**, **duration**, **next** and **prev**. The list is doubly linked: next points forward toward the tail and prev points back toward the head. The list is identified by its head pointer, which is NULL when the list is empty.

2. Algorithms

createNode(name, duration)

```
createNode(name, duration)
```

```
newNode <- allocate new Node
newNode.name <- name
newNode.duration <- duration
newNode.next <- NULL
newNode.prev <- NULL
return newNode
```

insertEnd(head, name, duration)

```
insertEnd(head by reference, name, duration)
```

```
newNode <- createNode(name, duration)
if head == NULL:
    head <- newNode
    return
temp <- head
while temp.next != NULL:
    temp <- temp.next
temp.next <- newNode
newNode.prev <- temp
```

displayForward(head)

```
displayForward(head)
```

```
if head == NULL:
    print "List is empty."; return
temp <- head
while temp != NULL:
    print temp.name, temp.duration, "min"
    temp <- temp.next
```

displayBackward(head)

```
displayBackward(head)
```

```
if head == NULL:
    print "List is empty."; return
temp <- head
while temp.next != NULL:        # walk to tail
    temp <- temp.next
while temp != NULL:
    print temp.name, temp.duration, "min"
    temp <- temp.prev
```

mergeLists(head1, head2)

```
mergeLists(head1, head2)
```

```
if head1 == NULL: return head2
if head2 == NULL: return head1
temp <- head1
while temp.next != NULL:        # find tail of list 1
    temp <- temp.next
temp.next <- head2
head2.prev <- temp
return head1
```

search(head, name)

```
search(head, name)
```

```
temp <- head
while temp != NULL:
    if temp.name == name:
        return temp
    temp <- temp.next
return NULL
```

deleteNode(head, name)

```
deleteNode(head by reference, name)
```

```
temp <- search(head, name)
if temp == NULL:
    print name, "not found."; return
if temp == head:                # deleting first node
    head <- temp.next
    if head != NULL:
        head.prev <- NULL
else:
    if temp.prev != NULL:
        temp.prev.next <- temp.next
    if temp.next != NULL:
        temp.next.prev <- temp.prev
delete temp
print name, "deleted successfully."
```

totalDuration(head)

```
totalDuration(head)
```

```
total <- 0
temp <- head
while temp != NULL:
    total <- total + temp.duration
    temp <- temp.next
return total
```

countCalls(head)

```
countCalls(head)
```

```
count <- 0
temp <- head
while temp != NULL:
    count <- count + 1
    temp <- temp.next
return count
```

deleteList(head)

```
deleteList(head by reference)
```

```
temp <- head
while temp != NULL:
    nextNode <- temp.next
    delete temp
    temp <- nextNode
head <- NULL
```

main()

```
main()
```

```
list1 <- NULL; list2 <- NULL
read n
repeat n times:
    read name, duration
    insertEnd(list1, name, duration)
read m
repeat m times:
    read name, duration
    insertEnd(list2, name, duration)
displayForward(list1); displayForward(list2)
list1 <- mergeLists(list1, list2)
displayForward(list1); displayBackward(list1)
read name
result <- search(list1, name)
if result != NULL: print caller name and duration
else: print name "not found!"
read name
deleteNode(list1, name)
displayForward(list1)
print countCalls(list1), totalDuration(list1)
deleteList(list1)
return 0
```

3. Complexity Summary

Function	Time	Notes
createNode	O(1)	Constant allocation and assignments
insertEnd	O(n)	Walks to the tail; O(1) if a tail pointer were kept
displayForward / displayBackward	O(n)	Visits every node once
mergeLists	O(n)	Walks list 1 to its tail, then links list 2
search	O(n)	Linear scan by name
deleteNode	O(n)	Dominated by search; unlinking is O(1)
totalDuration / countCalls	O(n)	One pass over the list
deleteList	O(n)	Frees every node once
Whole program	O(n + m)	Space O(n + m) for the nodes